

MyAnimeList Web Scraper

Detailed Code Breakdown & Presentation Guide

The Blueprint: Data Flow Architecture

This script functions as an automated data extraction pipeline consisting of four distinct stages:

[1. Target URLs] → [2. Raw HTML Download] → [3. Data Extraction] → [4. Clean Spreadsheet]

(Loop 0 to 150) (requests library) (BeautifulSoup) (pandas library)

1. Setup and Spoofing (Lines 1-10)

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
import time

headers = {
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)..."
}
anime_data = []
```

- **Library Imports:** Loading external tools required for network calls (`requests`), HTML parsing (`bs4`), data structuring (`pandas`), and execution delays (`time`).
- **Request Header Spoofing:** The `headers` dictionary mimics a legitimate user navigating via Google Chrome on Windows. This is a crucial scraping practice to bypass basic bot-protection mechanisms.
- **Data Structure Initialization:** `anime_data = []` sets up an empty list (array) to store the scraped dictionaries before conversion.

2. The Pagination Loop (Lines 14–18)

```
for offset in [0, 50, 100, 150]:  
    url = f"https://myanimelist.net/topanime.php?limit={offset}"
```

- **Dynamic URLs:** The scraper utilizes an f-string to inject the `offset` variable into the URL. This allows the script to navigate cleanly across four pages to gather 200 items without repeating code.

3. Network Requests & Initial Parsing (Lines 20–30)

```
response = requests.get(url, headers=headers, timeout=10)  
  
if response.status_code != 200:  
    continue  
  
soup = BeautifulSoup(response.text, "html.parser")  
rows = soup.find_all("tr", class_="ranking-list")
```

- **Robust Networking:** `requests.get` performs the HTTP GET request. The `timeout=10` prevents the script from hanging indefinitely during network failures.
- **Status Checks:** A status code of `200` equals "Success". Any other code triggers the `continue` statement, skipping the failed page and proceeding to the next.
- **DOM Traversal:** BeautifulSoup converts the raw HTML response into a navigable Document Object Model (DOM) tree. The script then isolates every table row (`<tr>`) belonging to the `ranking-list` CSS class.

4. Deep Data Extraction (Lines 32–70)

The code loops through each `row`, extracting raw text and applying data sanitization methods (like `.strip()` to remove whitespace).

```
info_tag = row.find("div", class_="information")  
lines = [x.strip() for x in info_tag.strip_strings]
```

```
anime_type = lines[0].split("(")[0].strip()
episodes = lines[0].split("(")[1].replace("eps)", "").strip()
```

- **String Manipulation:** Variables like Type and Episodes are embedded in a single messy string (e.g., "TV (64 eps)").
- `.split("(")`: Fractures the string into a list: `["TV ", "64 eps)"]`.
- `.replace("eps)", "")`: Deletes the trailing characters, sanitizing the input so that only the raw integer `"64"` remains.

5. Rate Limiting & Graceful Exits (Lines 72–78)

```
        time.sleep(2)
except KeyboardInterrupt:
    print("\nScraping stopped by user.")
```

- **Throttling:** `time.sleep(2)` limits the request rate to protect the target server from traffic overload.
- **Exception Handling:** Catching `KeyboardInterrupt` allows a user to stop the script midway without crashing, ensuring any already-scraped data is saved.

6. Serialization / Export (Lines 80–97)

```
finally:
    df = pd.DataFrame(anime_data)
    df.to_csv("top_anime.csv", index=False)
```

- **The `finally` Block:** Guarantees that the export code runs whether the script succeeds, encounters an error, or is manually stopped.
- **Pandas Serialization:** Converts the raw Python list of dictionaries into a structured tabular format (DataFrame) and exports it as a CSV file (`index=False` drops the row number column).



Presentation Cheat Sheet: Technical Terminology

Sprinkle these phrases into your presentation to demonstrate technical command over the project:

- **"DOM Traversal"**: Used when explaining how BeautifulSoup parses the HTML structure.
- **"Data Sanitization / Wrangling"**: Used to describe the string manipulation (`.split` and `.replace`) needed to clean up messy text strings.
- **"Robust Exception Handling"**: Used to refer to the `try/except` blocks that give the script high fault tolerance.
- **"Request Throttling"**: Used when pointing out the `time.sleep(2)` line.